Multi-agent systems

Architecture 5: University teaching deck

What changes when many agents work together, how are they organized and connected, and what can we learn from agent societies?

Patterns of orchestration, subagents as tools, CAMEL, MetaGPT, production frameworks, A2A, AI psychology, Smallville, and Deep Research systems.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Organizing agents

Define a multi-agent system and its benefits and costs. Compare centralized, decentralized, hierarchical, and federated patterns. Explain how a subagent can be treated as a tool.

Frameworks and protocols

Describe the role-playing idea in CAMEL and MetaGPT, name the production frameworks, and explain how A2A lets agents built on different platforms cooperate.

Societies and applications

Explain theory of mind and social identity bias in LLMs, describe the Smallville simulation, and outline the AI co-scientist and the Agent Laboratory as Deep Research systems.

Source: Ch. 8, PDF pp. 381–425

From one agent to many

Beyond the single agent

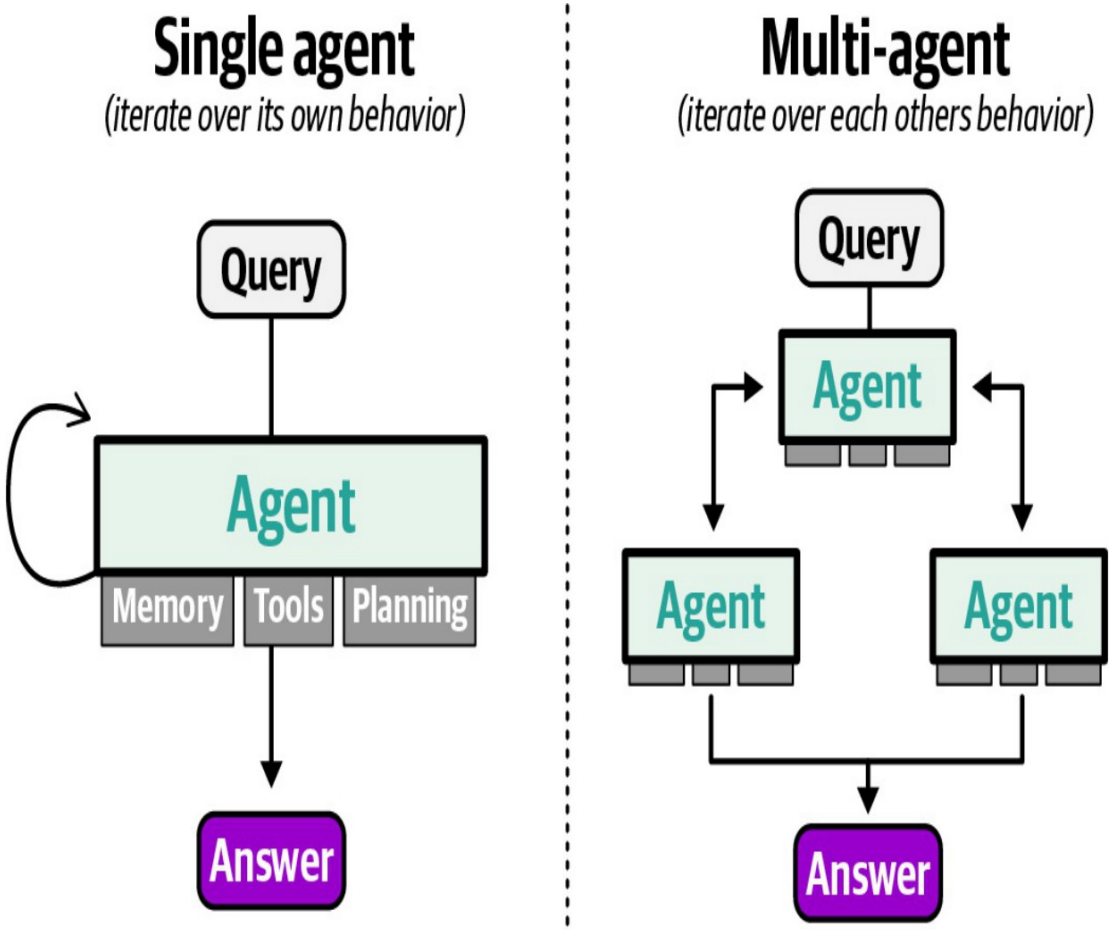
So far one agent has planned, executed, and reflected alone. In a multi-agent system (MAS), multiple agents collaborate, coordinate, or sometimes compete.

Shared goal, own goals

Agents work toward a common goal, and each may have its own goals. A search agent and a summarization agent have different roles and the same end goal.

Why split the work

A single research agent juggles searching, reading papers, critiquing, and summarizing. Specialist agents can each handle one of these tasks.



Grootendorst & Alammar (2026). Fig. 8-1, PDF p. 382

Environments and patterns in agentic systems

Shared or separate

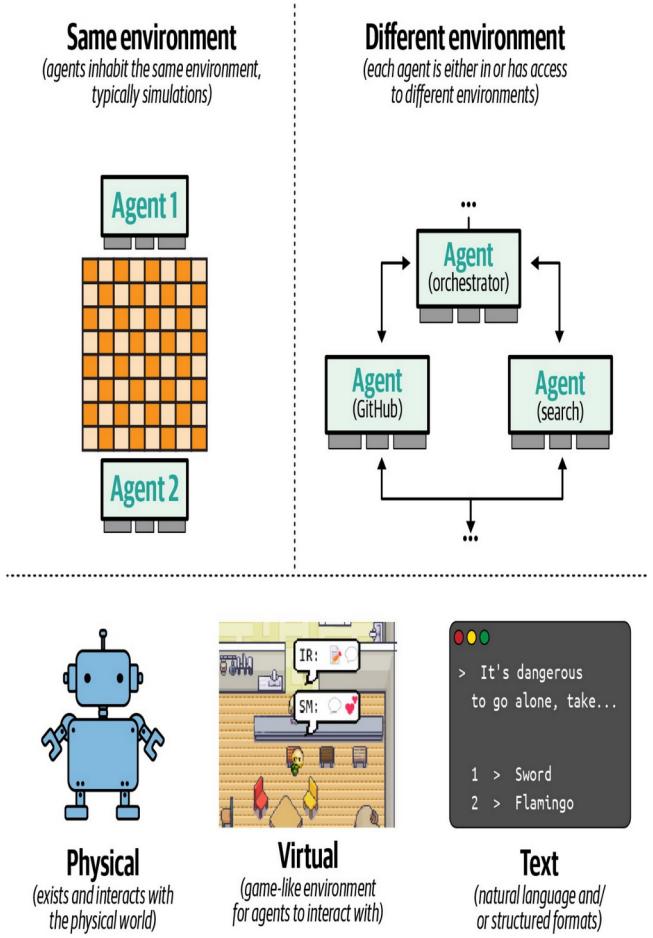
Agents may share an environment, like characters in a game, or occupy different ones. A booking system might have agents for places, flights, and scheduling.

Three kinds of environment

Physical (robots in the real world), virtual (game-like worlds), and text (such as coding IDEs or chat interfaces).

Scaling

A single-agent system has trouble scaling to large or complex use cases. A MAS can deploy more agents, each solving a different task.



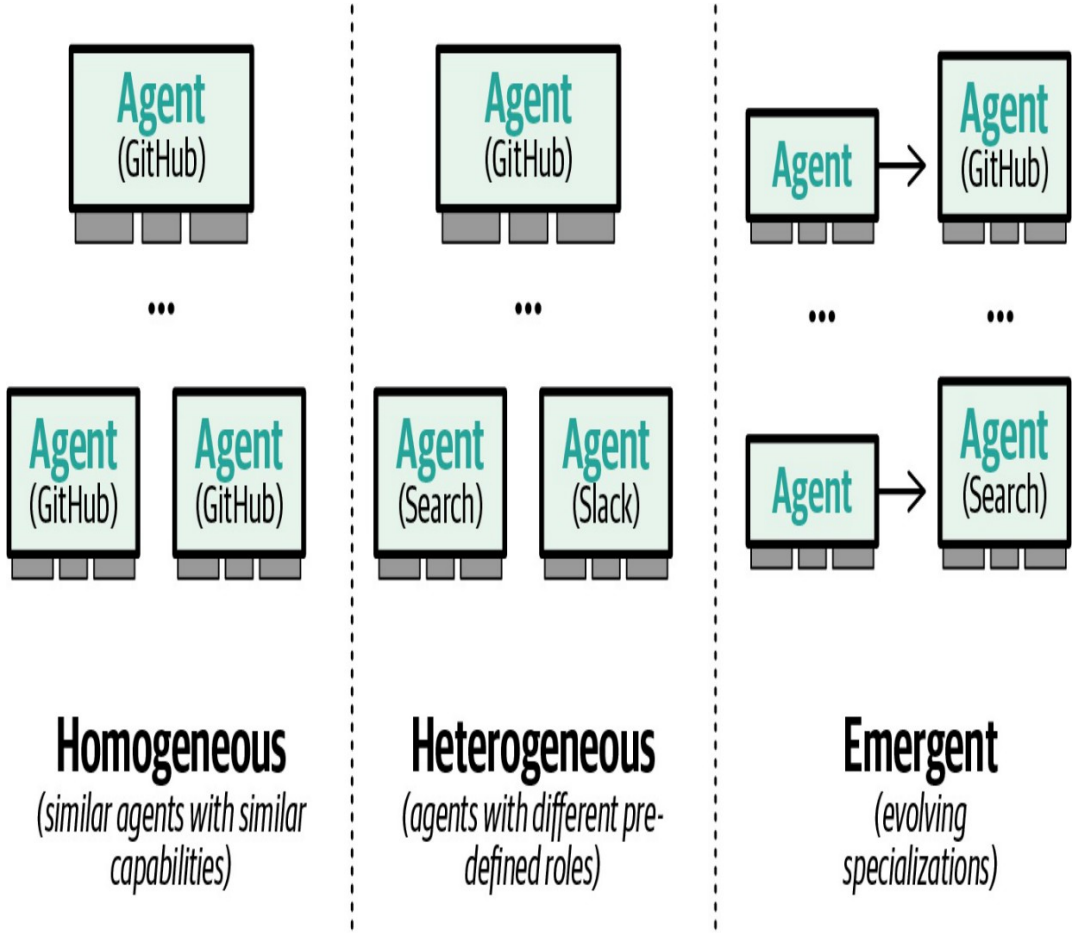
Grootendorst & Alammar (2026). Fig. 8-2, PDF p. 383

Agent roles in a system

Homogeneous
Agents have similar capabilities and are typically used for efficient parallel execution of tasks.

Heterogeneous
Agents differ in roles, each with its own goals and tasks. They may use different LLMs, memory systems, and tools.

Emergent specialization
Identical agents evolve into specialized roles through interaction, for example resource gathering or defensive roles in game-like simulations.



Grootendorst & Alammar (2026). Fig. 8-3, PDF p. 384

Benefits and costs of a MAS

Benefits

Collaboration solves diverse problems by adding agents. Scalability: agents can be added without slowing the system if they run in parallel or asynchronously. Speed: specialists work in parallel and tend to be faster.

Complexity

MASs need careful orchestration of agents and their interactions, and the interaction adds layers of abstraction.

Cost

More agents can mean more computation, especially if each agent relies on a powerful LLM.

Evaluation

Evaluating one agent is already tricky. Imagine evaluating many agents working together.

Examples of multi-agent systems in industry

Anthropic
Used multiple Claude agents to build a research system that lets users explore complex topics more effectively.

Uber
Built a data-retrieval MAS with a supervisor agent and several data-retrieval agents, such as an SQL agent, to turn user requests into real-time financial intelligence.

Delivery Hero
A food ordering and delivery company that uses several agents to extract entities and create product titles, building a product knowledge base.

Orchestration: four communication structures

Centralized
A single orchestration agent controls communication and allocates responsibilities and tasks to the other agents.

Decentralized
Each agent has similar responsibilities, and communication is distributed among them with no clear leader.

Hierarchical
Agents are structured in layers. Each level has a different degree of authority, and agents have distinct roles.

Federated
Agents are distributed among parallel environments or organizations that communicate only indirectly, to protect data privacy. Each follows its own regulations.

Four patterns of agent orchestration

Reading the figure

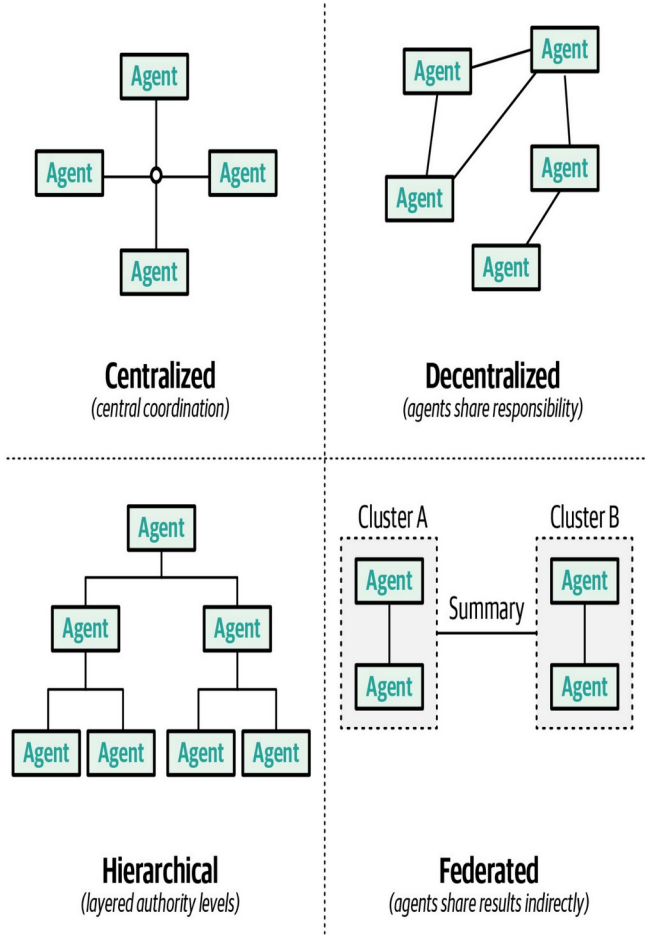
Centralized: a hub with agents around it. Decentralized: peers linked to each other. Hierarchical: layers of authority. Federated: clusters that share summaries.

Most used in practice

A centralized MAS is most often used because it is the easiest to implement and manage.

Design question

Who coordinates, who may talk to whom, and what happens when an agent fails? Each pattern answers differently.



Grootendorst & Alammar (2026). Fig. 8-4, PDF p. 387

Trade-offs of the four patterns

Centralized
Easy to build and manage. The entire system might collapse if the orchestration agent fails.

Decentralized
More stable, since the system continues if some agents fail, and scaling only needs more agents. It requires many capable agents and heavy communication.

Hierarchical
Intuitive, like a traditional organization, with efficient resource allocation through specialized roles. Still complex, and needs the most communication.

Federated
A relatively new pattern for agents across different organizations. It encourages cross-system collaboration but relies on shared standards that may not exist.

Subagents as tools

A neat trick

To build a centralized and potentially hierarchical system, view a subagent not as an agent but as a tool the orchestrator uses.

Two specialists

A math agent has add, subtract, and multiply tools. A date agent has tools to get today's date and to count the days between two dates.

Wrap as functions

Each subagent is wrapped in a function that passes a question to the agent and returns its answer. No complex framework is needed.

The orchestrator

The orchestrator receives those two functions as its tools and decides itself when to ask which specialist. This creates a simple hierarchy.

Grootendorst & Alammar (2026). Code, PDF pp. 388–390

Running the orchestrator

The question

"If I save €4 per day until 2030, how much will I have?" It needs both the number of days and a multiplication.

Step 1

The orchestrator asks the date agent how many days remain until the end of 2030. The answer is 1,691 days from the date of the run.

Step 2

It asks the math agent for €4 times 1,691, which returns €6,764.

Step 3

It answers that you will have €6,764. The answer depends on the run date, so a later run gives a different number.

Inspect the subagents, not only the orchestrator

An abstract overview

The orchestrator's trajectory shows only its own actions. It does not show what the subagents did or whether their routes were efficient.

The date agent's trajectory

It first called the tool for today's date, then the tool for days between two dates, and then answered. Reading it shows how well optimized the subagent was.

When it pays off

An orchestrator with dozens or hundreds of tools is likely to make mistakes while sifting through them. Giving a subset of tools to each subagent reduces that risk.

General-purpose frameworks and CAMEL

No single best pattern

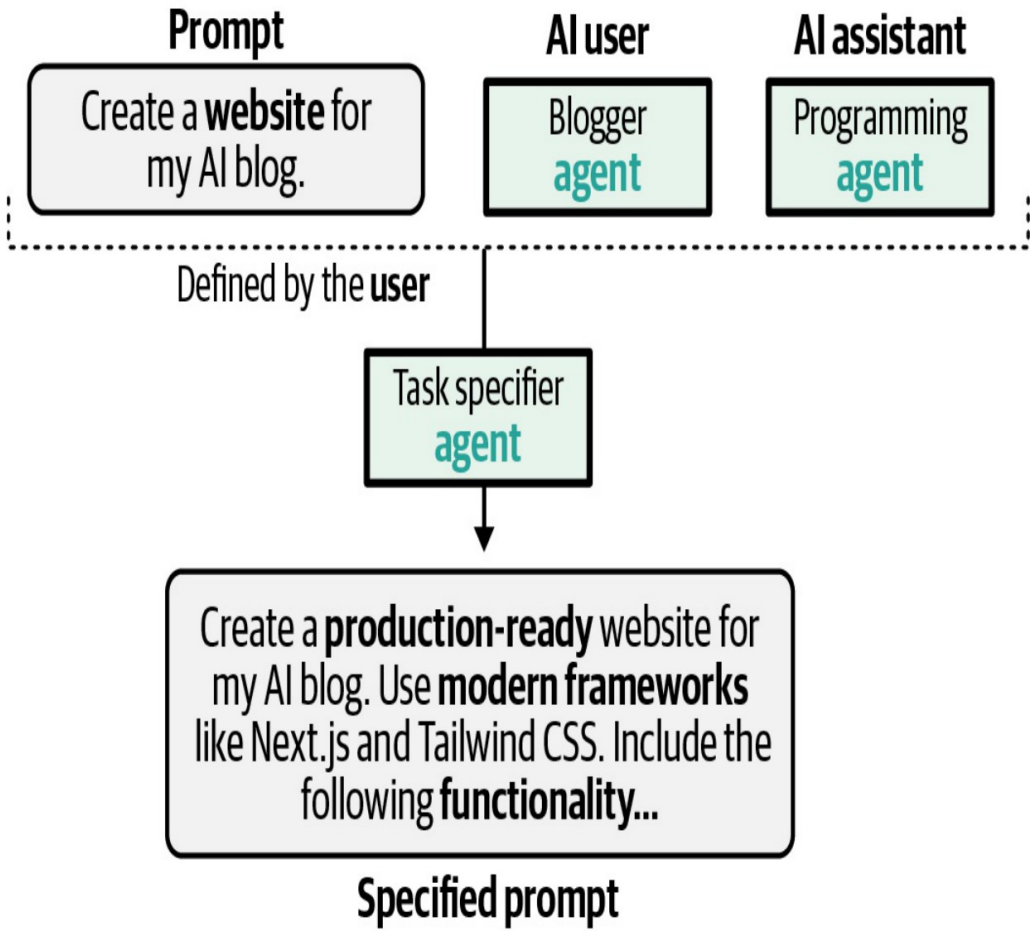
No framework, pattern, or workflow is best for all use cases, and each framework has its own flavor and philosophy.

CAMEL

One of the first collaborative MAS frameworks. Agents role-play to become specialized entities that collaborate.

User idea to roles

The user gives an idea, such as a website for a blog. Two roles are created, an AI user (a blogger) and an AI assistant (a programmer).



Grootendorst & Alammar (2026). Fig. 8-5, PDF p. 392

CAMEL: the user and assistant converse

Task specifier

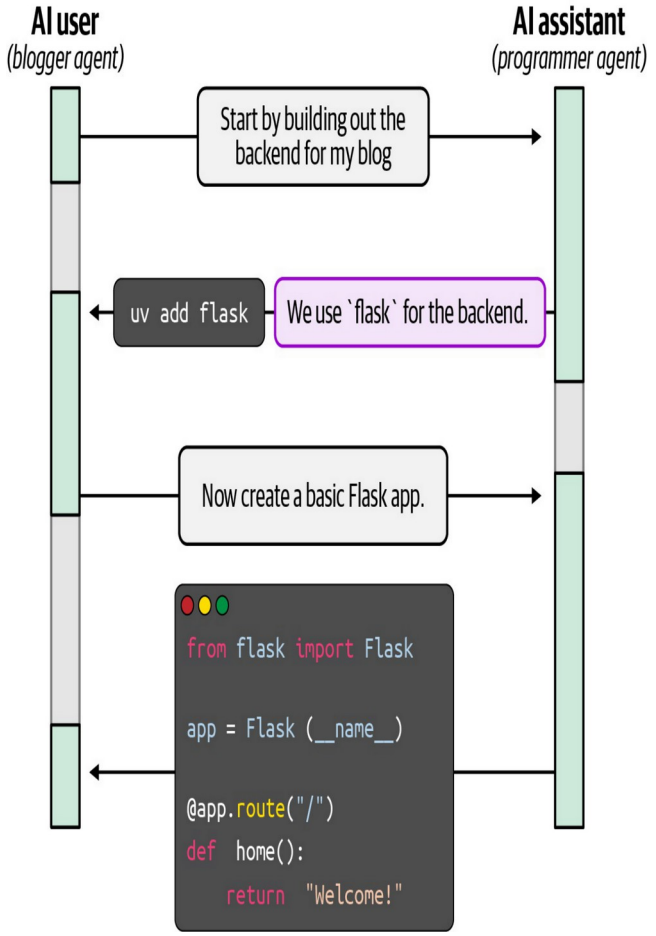
It receives the idea and the roles and rewrites the query into something more aligned with the task, with more detail.

Multi-turn conversation

The AI user gives instructions and the AI assistant executes them, over several turns of dialogue.

Stopping

The conversation ends when the AI user ends it and returns a completed answer. The structure is like role playing.



Grootendorst & Alammar (2026). Fig. 8-6, PDF p. 393

MetaGPT: mimicking an organization

Specialization of roles
Tasks are broken into smaller ones handled by specialized agents. Each role has a profile with a name, goal, skills, and constraints, modeled on, for example, a software company.

Communication protocol
A protocol for efficient communication with structured interfaces. Agents share artifacts such as documents and diagrams instead of communicating mainly through dialogue.

Executable feedback
Each agent iterates on its output to self-correct. An engineer writes code, runs it, and debugs it if needed. Agents follow a standard operating procedure and work sequentially, each powered by ReAct.

A MetaGPT pipeline of agents

From idea to code

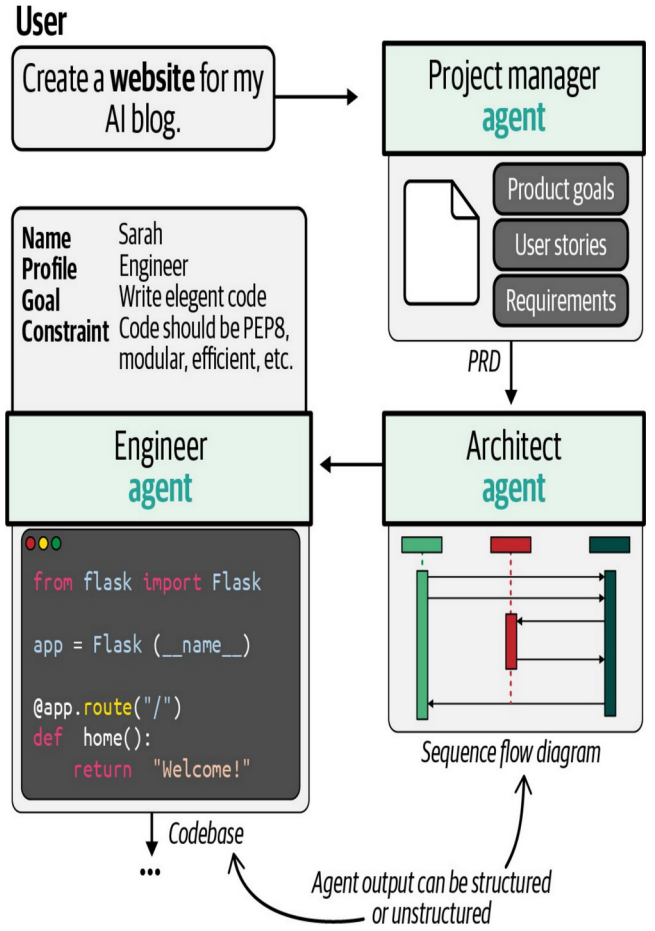
A user's request, such as a website for an AI blog, passes through a project manager, an architect, and an engineer agent.

Artifacts

The project manager produces product goals, user stories, and requirements. The architect produces a design. The engineer produces the codebase.

Profiles and checks

Each role has a profile, for example an engineer who writes elegant, PEP8-compliant code. The engineer's code is run and debugged.



Grootendorst & Alammar (2026). Fig. 8-7, PDF p. 395

Production-grade frameworks

Built for production

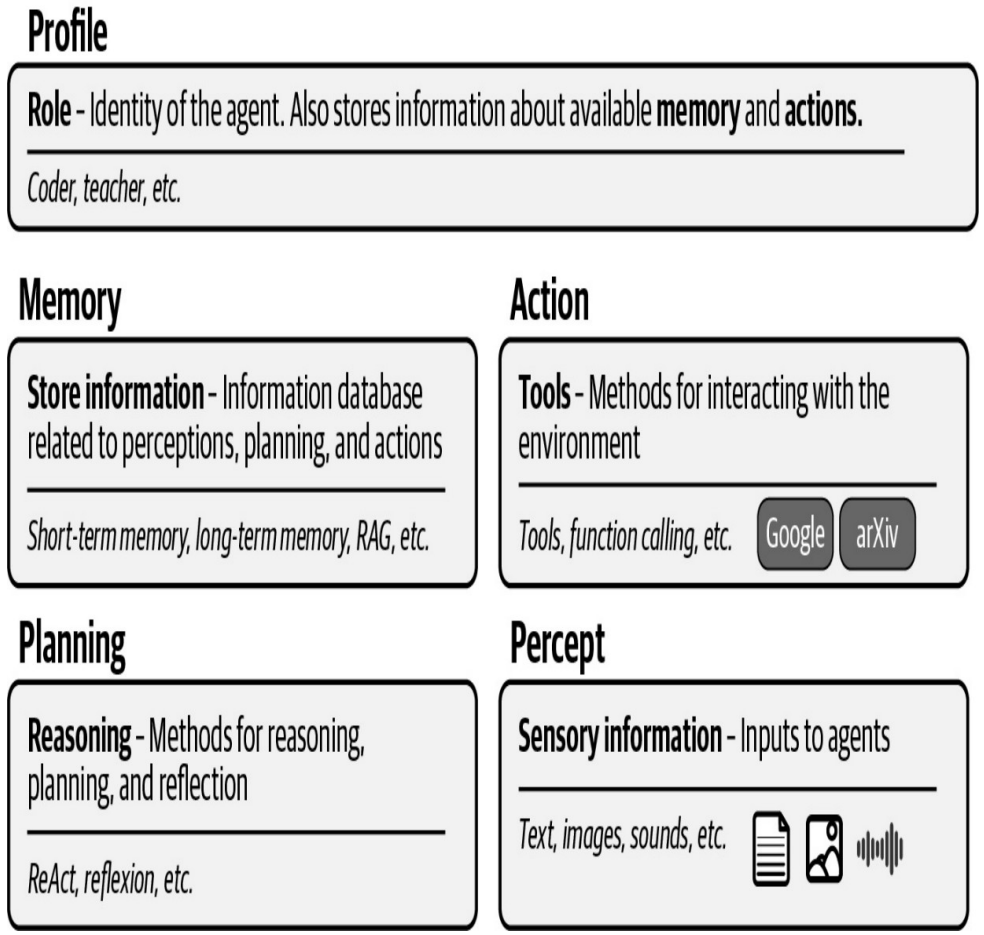
CAMEL and MetaGPT came out of research papers. The next group was made by developers to build and scale MASs in production.

Common at end of 2025

Microsoft's AutoGen, LangGraph, and CrewAI. Each is modular and lets you build any MAS you like.

What you configure

Roles, memory modules, tools for each agent, and the pattern of the system, as the figure illustrates.



Grootendorst & Alammar (2026). Fig. 8-8, PDF p. 397

Choosing a framework

Experiment
With a constantly changing landscape of AI agents, it is difficult to suggest a single best platform. Try candidates on your own task.

Independence and continuity
Consider how independent a platform is and whether its development will continue. These matter as much as features.

n8n
An open source AI workflow automation tool. You define workflows that LLMs and agents follow, with agents still holding some autonomy. It is on the edge of a MAS.

Why communication protocols matter

Format affects performance

How agents exchange information greatly affects performance. As Chapter 4 explained, context engineering needs both inputs and outputs optimized. Too much, too little, or the wrong format degrades results.

Before LLMs

Agent communication languages such as FIPA-ACL and KQML standardized message types like request and inform.

Since LLMs

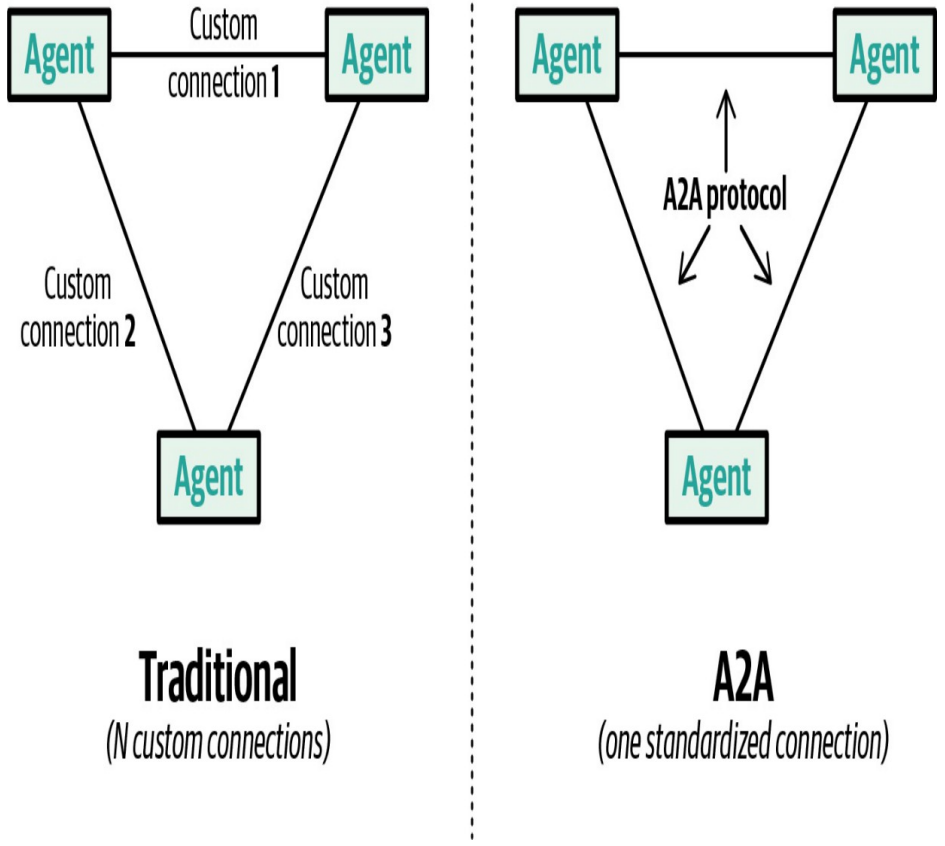
Since GPT-3.5 in late 2022, protocols connect agents and tools, such as MCP in Chapter 5. A collaborative MAS would greatly benefit from a standard for agent-to-agent communication.

A2A: standard agent-to-agent communication

Open standard
Agent2Agent (A2A), first developed by Google, lets agents built on different frameworks and by different organizations work together.

The problem
Agents made with LangGraph, CrewAI, or AutoGen on Azure, AWS, or Google Cloud would each need custom connections to one another.

The solution
A2A standardizes the connection, so each agent implements the protocol once and the number of custom connections falls.



Grootendorst & Alammar (2026). Fig. 8-9, PDF p. 399

MCP and A2A together

Two layers

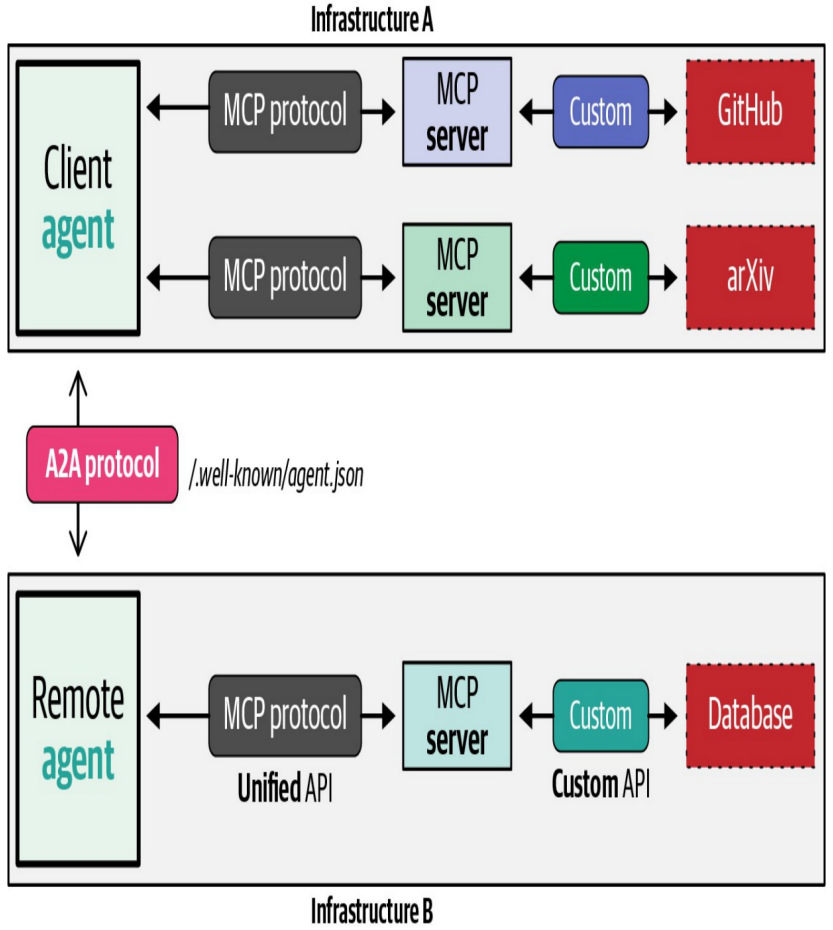
MCP connects an agent to its tools and data. A2A connects an agent to other agents. The figure shows both in one system.

Different infrastructure

Two agents that each have access to MCP tools are hosted on different infrastructures and communicate through A2A.

What A2A allows

Discover other agents and their capabilities, exchange structured or unstructured information across modalities, stream responses, and hold multi-turn conversations.



Grootendorst & Alammar (2026). Fig. 8-10, PDF p. 400

A2A actors and the five steps

Three actors

The user (usually a human, possibly an automated process) defines a goal. The client agent acts for the user. The remote agent (A2A server) executes the task and may be a single agent or a whole MAS.

Steps 1 and 2

Task initiation: the user gives the client agent a task, with no A2A needed yet. Discovery: the client finds which agents exist and what they can do.

Steps 3 and 4

Authentication through the scheme in the agent card, such as API keys or OAuth 2.0. Then the client sends a request over HTTPS with JSON-RPC 2.0, much like MCP.

Step 5

The remote agent finishes and sends a message back with relevant information or artifacts. It may ask for more information and stream updates while working.

The agent card for discovery

A JSON file

Every agent exposes a file describing its capabilities, endpoint URL, and other information. This is the agent card.

Well-known location

It is available at a standard path, /.well-known/agent-card.json, so a client can find it without prior arrangement.

An example: a train agent

The card of a train agent gives its name, description, URL, version, capabilities such as streaming and push notifications, and its input and output modes.

Skills and examples

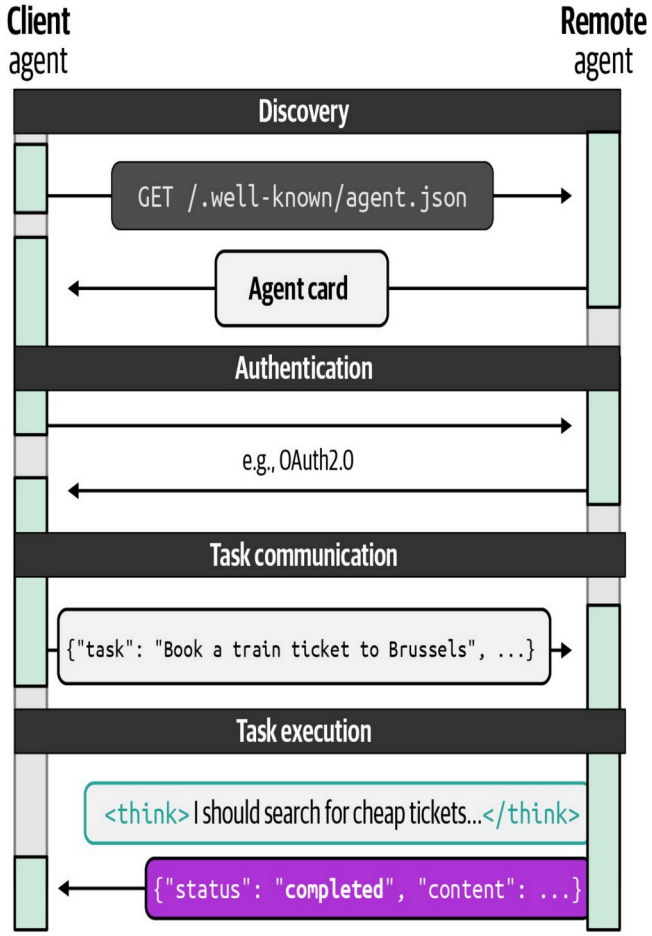
The card lists skills with an id, name, description, tags, and examples, such as booking return tickets between two cities on a given date.

The A2A flow between client and remote agent

Discovery
The client fetches the agent card from the well-known address and learns what the remote agent offers.

Authentication
The client authenticates according to the scheme in the card, such as OAuth 2.0.

Task communication and execution
The client sends a JSON task. The remote agent thinks, calls its tools, and returns a status message with its content.



Grootendorst & Alammar (2026). Fig. 8-11, PDF p. 403

Other approaches and open directions

Few competitors

With a technology as new as A2A there are not many competitors.

Internet of Agents (IoA)

An agent integration protocol for creating MASs, based on the concept of the internet. It has an instant-messaging-like design and dynamic modules for agent conversation and collaboration.

Reinforcement learning for MAS

RL can train LLMs and agents inside a MAS, so that they are already experienced with such systems and standardization and collaboration improve.

From collaboration to agent societies

Agentic societies

As agents collaborate and interact, the first glimpses of agentic societies appear: simulations of interactive artificial societies run by agents.

A common environment

Agents interact in a shared environment without always having specific tasks to complete.

What can emerge

Sociality, identity, and potentially theory of mind. The section looks at the emergence of individual and social identities and behavior in social simulations.

AI psychology: identity and emotional intelligence

Identity by prompt

An LLM can be given an identity in its system prompt, for example a name and a profession. At first this is an individual portrayal that social interaction does not guide.

Human data, human traits

LLMs are trained on human-generated data and mimic our behavior, so some human psychology applies to them. This field is called AI psychology.

Emotional intelligence

Beyond cognitive ability, LLMs show some capacity to recognize, understand, and express emotions.

Dependency

One study suggests users with an anxious attachment personality are predisposed to emotional dependency on GPT-4.

Theory of mind

The concept

Theory of Mind (ToM) is the ability to attribute mental states, such as emotion, to others. It lets us understand that others have desires and beliefs different from ours, and it gives rise to empathy.

Why it matters

Different types of intelligence may arise from interaction and not from isolation. Research shows early indicators of ToM in LLMs.

The Reddit study

Authors used the Change My View community to source open-ended discussions. LLMs wrote reasoned responses. Human judges scored their reasoning and fit with the poster's intent.

Social identity bias in LLMs

A human bias

Social identity bias is the tendency to favor our own group and be hostile to others. Research by Hu and colleagues (2025) found it is also a property of LLMs.

The method

Seventy-seven LLMs were given in-group sentence starts ("We are") and out-group starts ("They are"), and completed them.

The result

In-group sentences tended to be more positive than out-group sentences. Because LLMs learn from our behavior, it is not surprising that they mimic it.

Measuring sentiment

Sentiment of the completions was measured with RoBERTa and VADER, as the book's Table 8-1 shows, with examples ranging from positive to negative.

Ethical considerations

Anthropomorphism

With seemingly strong emotional capabilities, it is hard not to anthropomorphize LLMs. It is not yet clear whether these are accurately mimicked behavior or something more fundamental.

Critical settings

Using LLMs in situations such as mental healthcare and suicide prevention raises ethical questions. If something goes wrong, should the LLM be held responsible?

Open questions

Is the service provider responsible if the model was trained with flaws or under-evaluated? Are the model's creators responsible if the provider misuses it? What counts as misuse? The authors do not answer but stress the need for ethical consideration.

Simulations as world models: Smallville

World models

If a simulation is accurate enough it can represent the mechanisms of the world it simulates, and is then called a world model. In MAS they explore social behavior.

Generative Agents

One of the most influential MAS papers puts agents in a pixel sandbox where they plan days, join group activities, and form relationships.

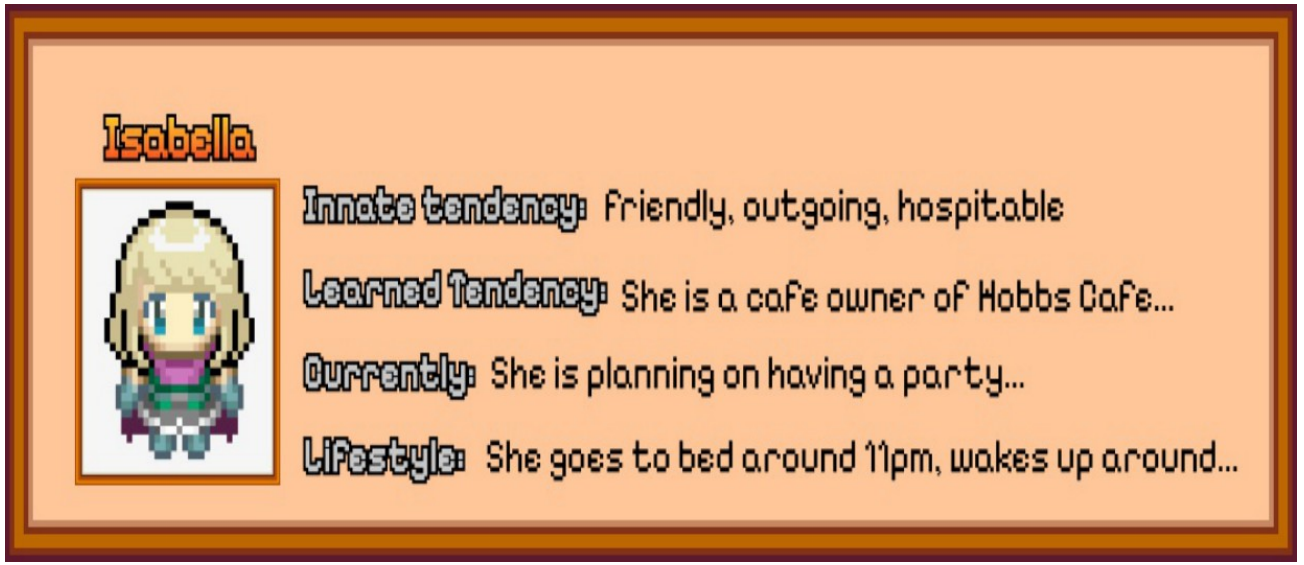
Smallville

The simulated small town of the paper, shown in the figure adapted from it.



Grootendorst & Alammar (2026). Fig. 8-12, PDF p. 408

Agent profiles: Isabella Rodriguez



Distinct identities
Each agent has a profile that makes it behave in unique ways. The figure shows Isabella Rodriguez, who owns Hobbs Cafe, with her traits, tendencies, and current state.

Twenty-five agents
The authors created 25 agents, each initialized with one paragraph on identity, occupation, and relationships.

Three modules
Each agent had memory, planning, and reflection, like the core components of ReAct and Reflexion in Chapter 6.

Grootendorst & Alammar (2026). Fig. 8-13, PDF p. 409

The memory stream

Not only summaries

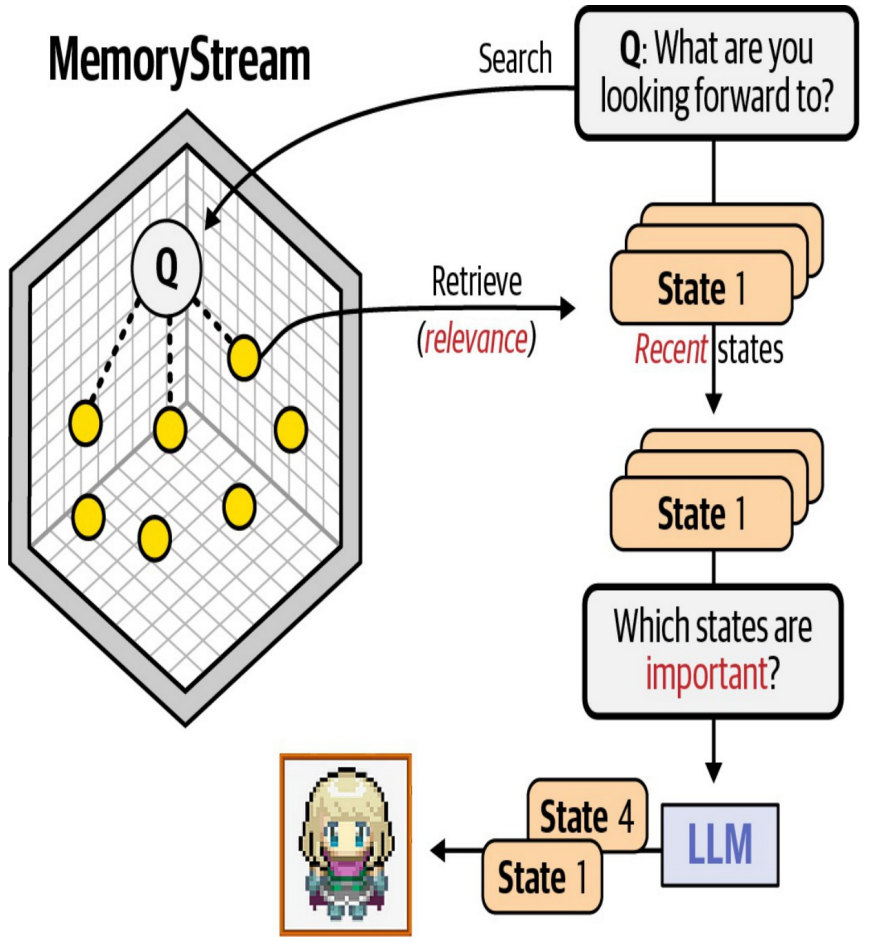
Summarization reduces overload but may summarize irrelevant information. The memory stream scores each state on recency, importance, and relevance.

Three attributes

Recency gives higher scores to recent states. Importance is judged by another LLM, asking how important the state is to the agent. Relevance is the cosine similarity between a question and an embedded state.

RAG-like

A large number of timestamped states, such as a desk being idle, is distilled into the few relevant to the current situation.



Grootendorst & Alammar (2026). Fig. 8-14, PDF p. 410

Reflection makes behavior more lifelike

A second memory type

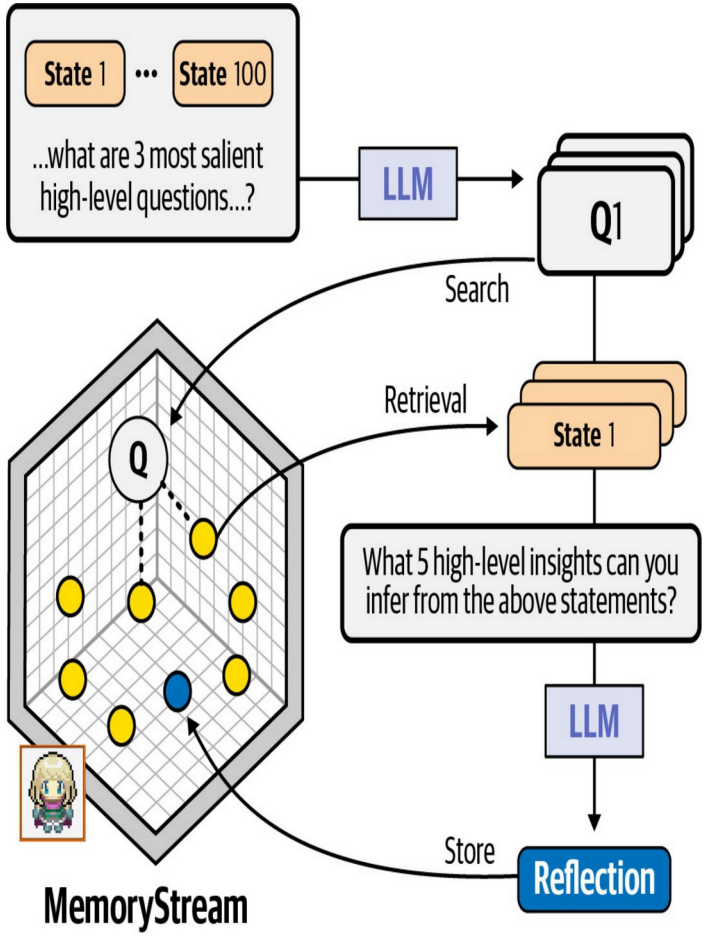
Agents periodically reflect on recent events, roughly two or three times a day.

The procedure

Retrieve the 100 most recent states, prompt an LLM for the three most salient high-level questions, retrieve states relevant to them, and answer with five insights.

Fed back

The reflections are put back into the memory stream and can themselves be reflected on.



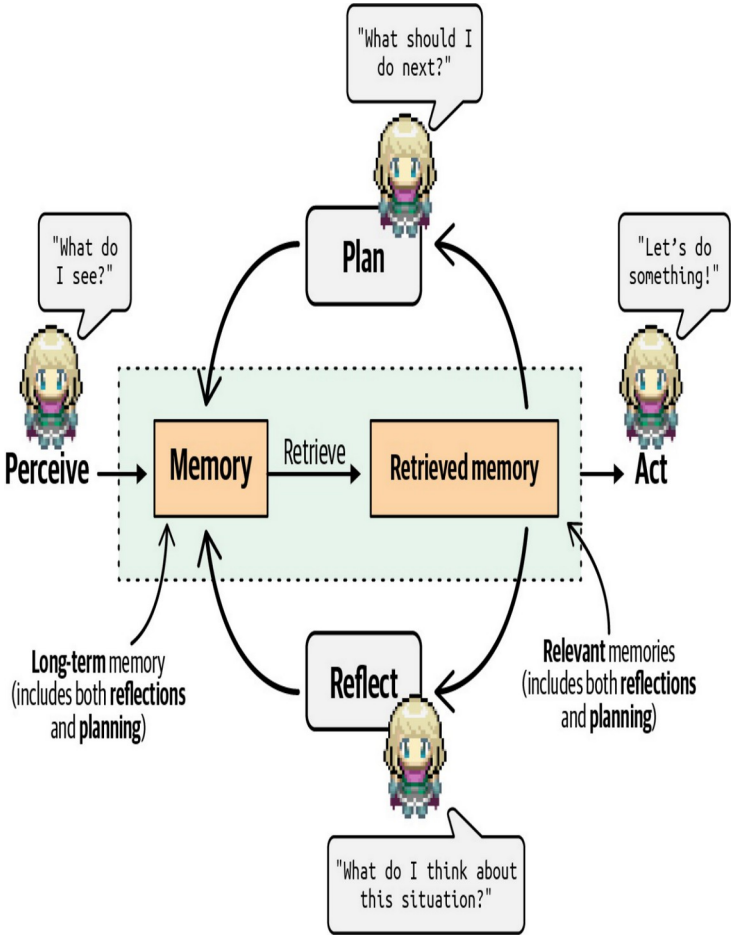
Grootendorst & Alammar (2026). Fig. 8-15, PDF p. 412

Planning and reacting in the agent loop

Plans
Agents create and continuously update plans with a location, a starting time, and a duration. Plans are stored in the memory stream like reflections.

Decide with all of it
To choose the next action, agents consider observations from the environment, reflections, and plans.

A ReAct-like schema
If an observation is important, the plan is updated. The LLM is asked whether the agent should react to the observation and how.



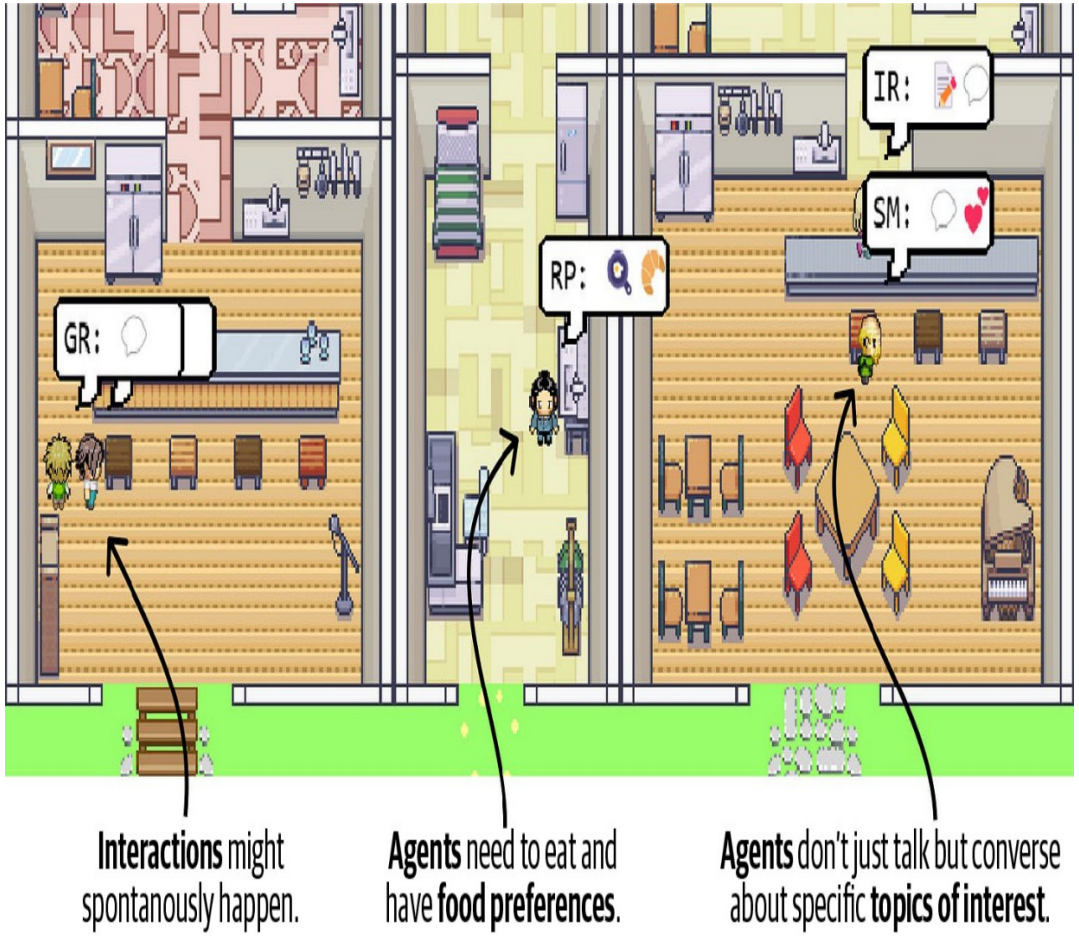
Grootendorst & Alammar (2026). Fig. 8-16, PDF p. 414

Behaviors that emerge in Smallville

Dialogue
Agents interact through dialogue, which can create spontaneous interactions based on their memory of the agents they meet.

Conversations on topics
They may converse about topics of interest. The figure, annotated from the interactive demo, shows different types of behavior.

Worth reading
The authors of the book urge readers to read the paper. It is a good example of the single agent structure with added social interaction and communication.



Grootendorst & Alammar (2026). Fig. 8-17, PDF p. 415

Other simulacra: Agent Hospital and SimClass

Beyond social life

Since Smallville, many simulacra have served purposes other than purely social interaction.

Agent Hospital

Inspired by Smallville, it is a hospital sandbox that simulates the whole patient journey, from disease onset and triage to the dispensary and the follow-up visit. Doctor agents evolve as they treat many patient agents.

SimClass

Simulates classroom education with agent-student and student-student interaction. Emerging behavior shows student agents working together to solve tasks.

Deep Research agents

What it is

A system in which agents work together to research topics and write comprehensive reports using various resources. In 2025 Anthropic, Perplexity, Google, and OpenAI adopted Deep Research products.

Single and multi-agent

Single agents such as Search-o1 and Search-R1 appeared in earlier chapters. MASs fit Deep Research because of its complexity and collaborative nature.

Anthropic's example

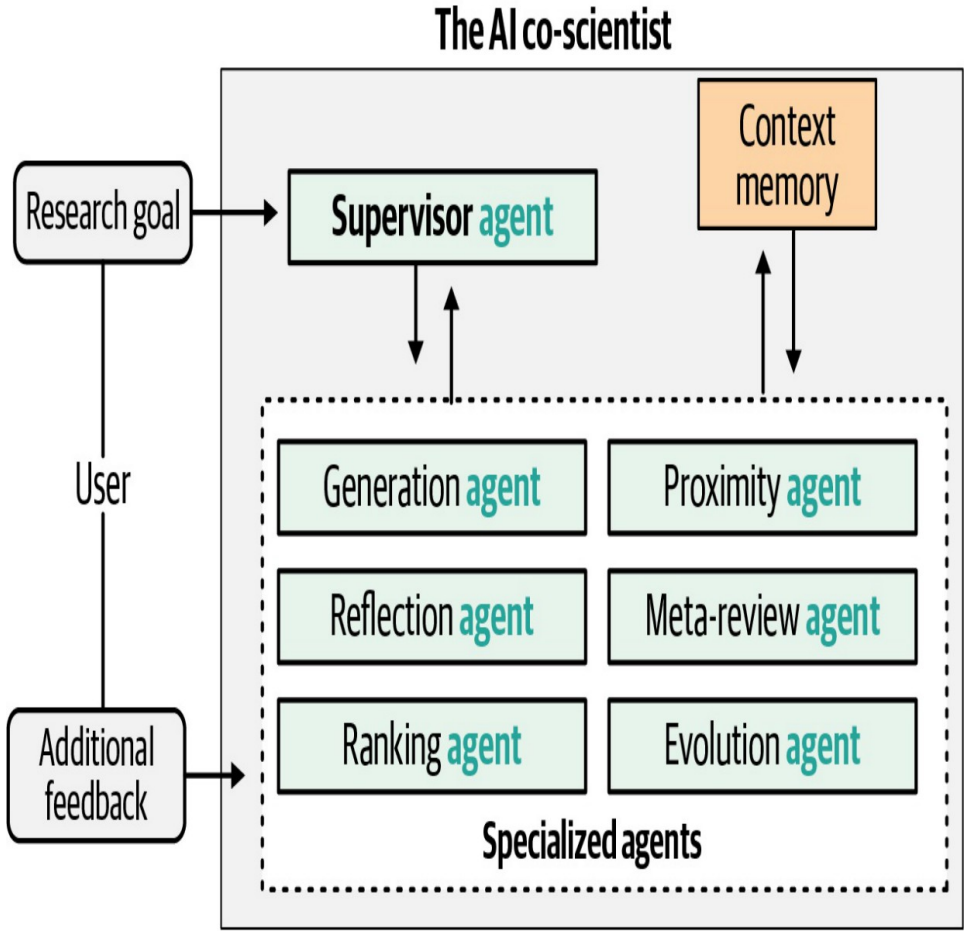
Its multi-agent research system has search agents, a citation agent, and a lead researcher agent that orchestrates the workflow.

The AI co-scientist

Aim
Released in February 2025 to help scientific discovery, it generates hypotheses and a research proposal from a research goal. Built on Gemini 2.0, it follows the scientific method.

Criteria
Hypotheses should be plausible, novel, testable, and safe, and the criteria can be adjusted by the user.

Supervisor
The user gives a goal and documents. A supervisor agent derives a plan and orchestrates specialized agents, with all state in a context memory.



Grootendorst & Alammar (2026). Fig. 8-18, PDF p. 418

The co-scientist's specialized agents

Generate, reflect, rank

The Generation agent starts with literature searches for focus areas. The Reflection agent checks correctness, quality, and explanatory power. The Ranking agent runs Elo-style tournaments through scientific debates.

Group, evolve, review

The Proximity agent finds similar hypotheses so they can be grouped and de-duplicated. The Evolution agent refines the best ones. The Meta-review agent combines insights from reviews and debates to improve the system.

Human in the loop

The scientist can guide the system while it works: review hypotheses, evaluate proposals, ask it to follow up on one, or prioritize fields. It empowers people through human and machine collaboration.

Agent Laboratory, phase 1: literature review

A fixed pipeline

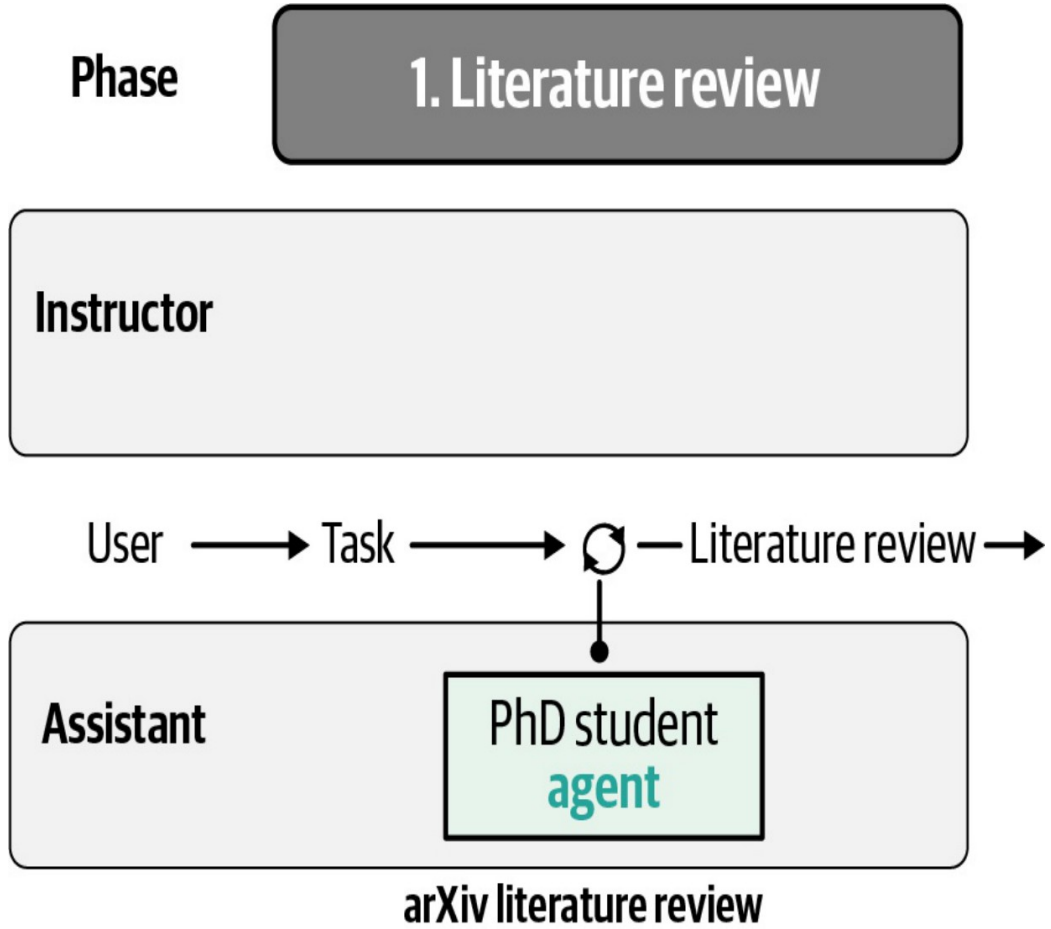
A more static system than the co-scientist, with predefined stages: literature review, experimentation, and report writing. The human researcher drives it.

Goal

To produce a research report together with the code needed to run the experiments, reducing time-intensive work such as coding and documentation.

Phase 1

A PhD student agent uses the arXiv API to find papers, refines its query iteratively, and stops at a set number of relevant papers.



Grootendorst & Alammar (2026). Fig. 8-19, PDF p. 419

Agent Laboratory, phase 2: experimentation

Plan formulation

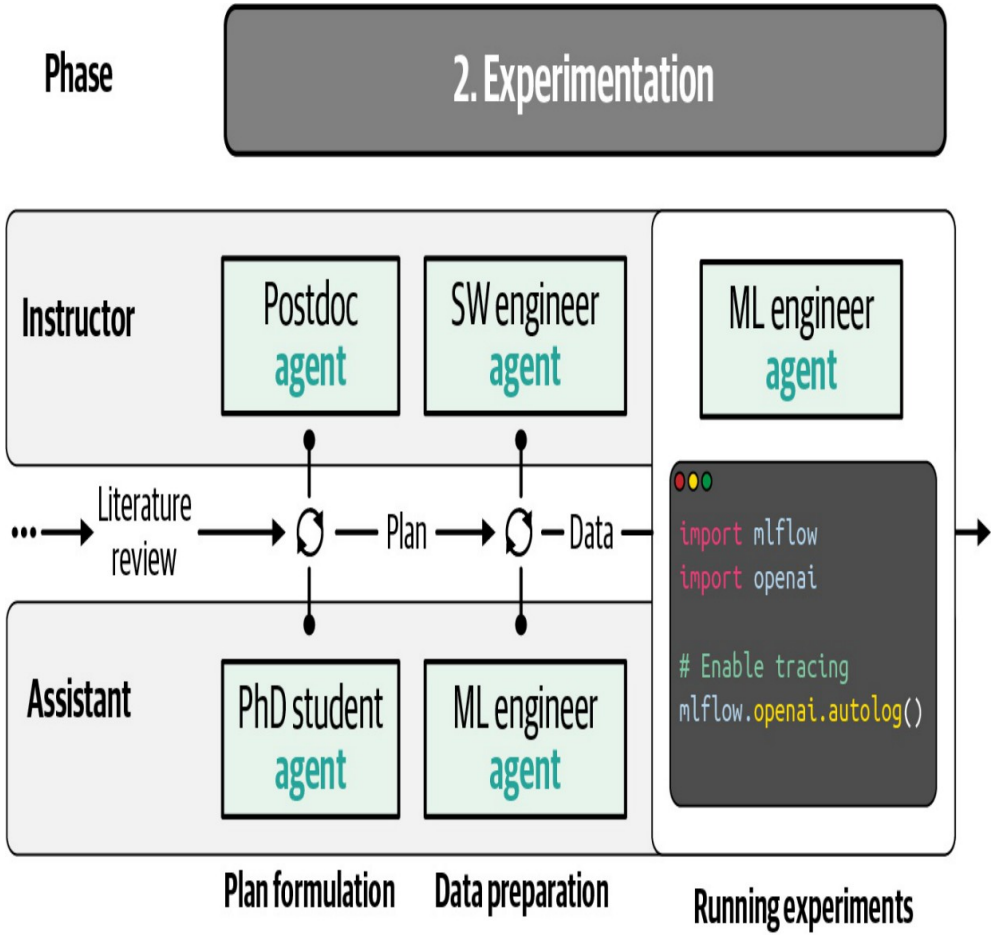
A PhD student agent and a postdoc agent produce a research objective and the experimental steps in a plan.

Data preparation

A machine learning engineer agent searches the HuggingFace datasets library for the best data. A software engineer agent submits the code and checks for bugs.

Running experiments

The ML engineer implements the plan with mle-solver, an advanced coding agent (see Chapter 10) that generates the code autonomously.



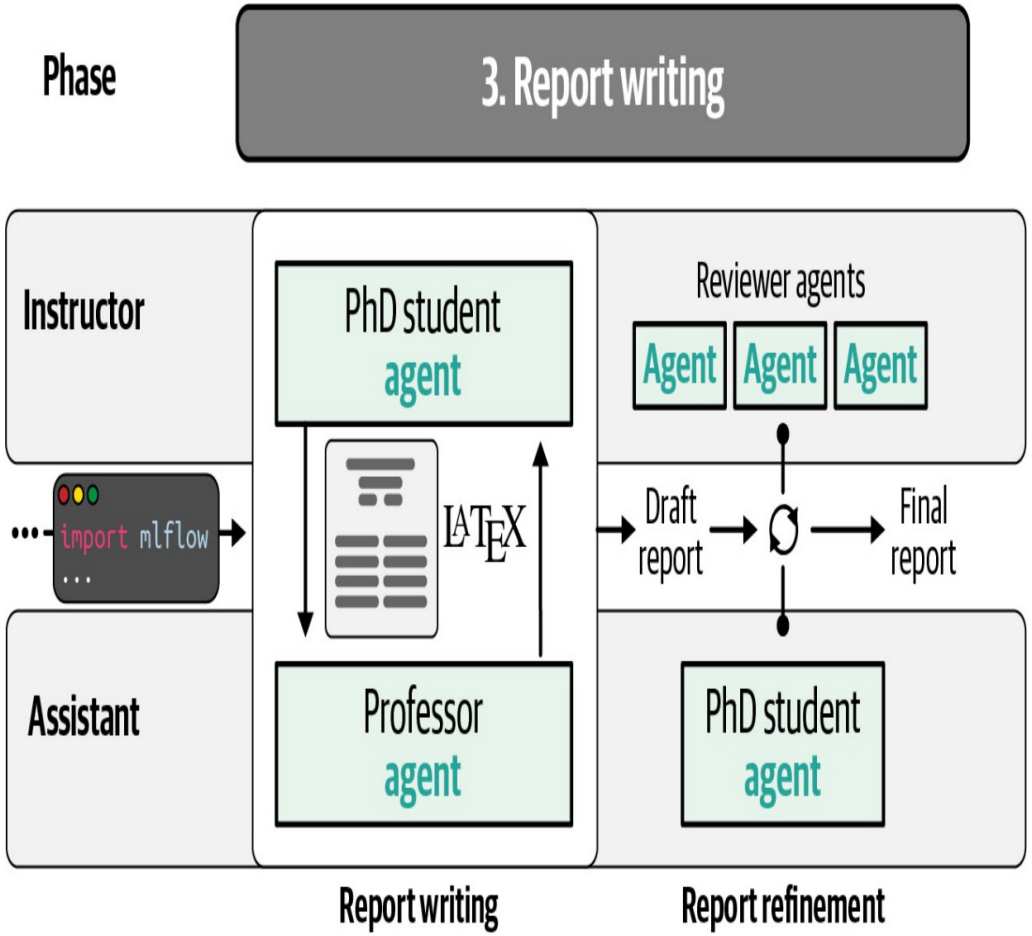
Grootendorst & Alammar (2026). Fig. 8-20, PDF p. 421

Agent Laboratory, phase 3: report writing

Report writing
The PhD student and professor agents turn the findings into an academic report, with frequent reviews by the professor agent and extra arXiv research.

Report refinement
Three reviewer agents, based on NeurIPS peer reviewers, rate the draft on originality, quality, clarity, and significance.

Loop until good
The PhD agent decides whether to address feedback or finish. The report improves until all agents agree it is of sufficient quality.



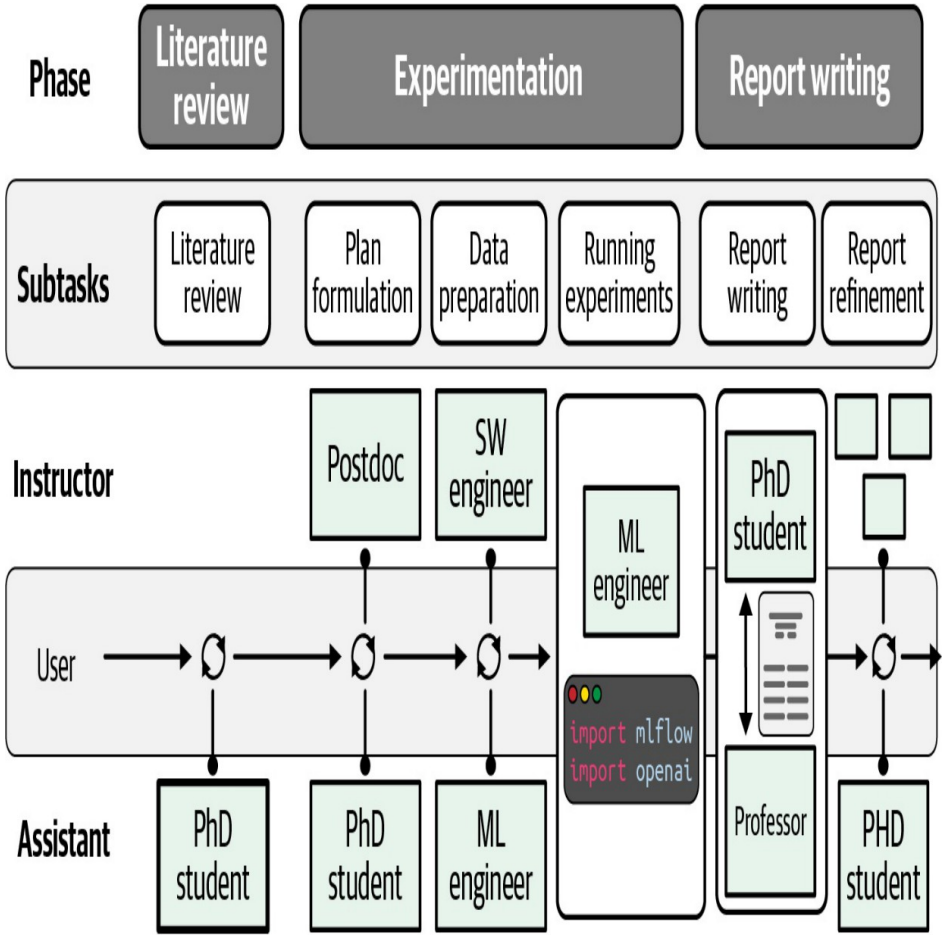
Grootendorst & Alammar (2026). Fig. 8-21, PDF p. 422

Agent Laboratory: all phases

Mirrors academic work
Literature review, experimentation, and report writing mimic a real-world process of academic exploration and revision.

Fixed order, free steps
The order of steps is fixed, so the architecture has no autonomy. Within each step, agents iterate with a high degree of autonomy until satisfied.

Compare with co-scientist
The co-scientist's supervisor chooses dynamically which agent to run. The Agent Laboratory follows a fixed pipeline.



Grootendorst & Alammar (2026). Fig. 8-22, PDF p. 423

Choosing an orchestration pattern

A help desk

One front-line agent routes requests to specialist agents. Centralized fits: it is easy to manage, and the router is the risk to protect.

A research team

Layers of agents with a lead, workers, and reviewers. Hierarchical fits the way organizations divide authority, at the price of communication.

A swarm of equals

Many similar agents run in parallel on parts of a big job. Decentralized adds resilience but needs many capable agents.

A hospital network

Agents in separate organizations that cannot share raw data. Federated fits, if standards for sharing results exist.

Teaching addition. Concepts: Ch. 8, PDF pp. 386–388

Multi-agent systems and accountability

Who acted?

With a chain of agents, an outcome comes from many decisions. Logs of each agent's trajectory are needed to reconstruct who did what.

Where is the authority?

In a hierarchy, authority can be written into the design. In a decentralized system there is no single point of responsibility, so it must be assigned another way.

Cross-organization trust

A2A lets agents from different providers cooperate. Authentication, agent cards, and the data shared across organizations then need review.

Responsibility for people

The book's questions about mental healthcare settings show that the model creator, provider, deployer, and user may all share responsibility.

Teaching addition. Concepts: Ch. 8, PDF pp. 385–407. Cross-reference: governance half of the course

Activity: design a small multi-agent system

The task

Design a system that prepares a weekly briefing about a topic for a department. Decide whether one agent or several are needed, and why.

The design

Name each agent, its role, tools, and memory. Choose a pattern and draw who talks to whom. Say what each agent hands over: a document, a list, or a message.

The risks

List what could go wrong: a failing orchestrator, cost, disagreement among agents. Say how you would evaluate the system, using ideas from Chapter 7.

Teaching addition. Concepts: Ch. 8, PDF pp. 384–423

Learning check with worked answers

1. Which pattern is most common?
Centralized, because it is the easiest to implement and manage. Its weakness is that the system may collapse if the orchestrator fails.

2. MCP or A2A?
MCP connects an agent to tools and data. A2A connects agents to other agents, including those from other frameworks and clouds.

3. What is an agent card?
A JSON file at a well-known path that describes an agent's capabilities, endpoint, skills, and security scheme, so clients can discover it.

4. Why do Smallville agents reflect?
Reflection turns many raw states into higher-level insights that are put back into memory, which makes behavior more consistent and lifelike.

Chapter summary

Orchestration

Going from one agent to a MAS starts with orchestration. Centralized, decentralized, hierarchical, and federated patterns; CAMEL, MetaGPT, AutoGen, LangGraph, and CrewAI; and A2A as the multi-agent counterpart of MCP.

Agent societies

How agents communicate gives insight into AI psychology: emotional intelligence, theory of mind, and social identity. The Smallville sandbox showed agents planning days and forming relationships.

Deep Research

A common use case for MAS. The Agent Laboratory and the AI co-scientist show how orchestration is implemented. The next chapter covers modalities other than text.

Core distinctions and source guide

Conceptual distinctions

A multi-agent system adds orchestration and communication to the single agent. The pattern (centralized, decentralized, hierarchical, federated) sets who coordinates. MCP connects agents to tools, and A2A connects agents to agents. Simulations study social behavior, and Deep Research systems apply the idea to real work.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 8. Selected connections: Chapters 4, 5, 6, 7, and 10.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF